

# Rélatório de IA – Problema das 8 Rainhas

Anderson Rodrigues dos Santos  
Maíra Beatriz de Almeida Lacerda

30 de outubro de 2025

## 1 Introdução

O problema das Oito Rainhas é um clássico da Inteligência Artificial e da otimização combinatória. O desafio consiste em posicionar oito rainhas em um tabuleiro  $8 \times 8$  de forma que nenhuma ataque outra. Nesta segunda parte do trabalho, foram implementadas duas variações do algoritmo Hill Climbing para resolver o problema das 8 Rainhas: uma com movimentos laterais e outra com reinício aleatório. O objetivo é comparar os resultados dessas abordagens em termos de tempo de execução, número de iterações, número de reinícios e taxa de sucesso.

## 2 Metodologia

### 2.1 Representação do Problema

O problema das 8 Rainhas foi modelado como um vetor de tamanho 8, onde cada índice representa a coluna e o valor armazenado na posição do vetor representa a linha da rainha naquela coluna.

- **Estado:** vetor de tamanho 8, onde cada valor no vetor é a posição da rainha naquela coluna.
- **Ações:** mover uma rainha de sua linha atual para outra linha da mesma coluna.
- **Transição:** o movimento é válido se não houver conflito entre as rainhas.
- **Custo:** custo unitário por movimento.
- **Teste de objetivo:** número de conflitos ( $h(n) = 0$ ).

A função heurística  $h(n)$  é definida como o número de pares de rainhas em conflito no estado atual. O objetivo é minimizar  $h(n)$ , atingindo  $h(n) = 0$ , que representa uma solução válida.

### 2.2 Implementação do Algoritmo de Hill Climbing

O algoritmo de **Hill Climbing** foi implementado com duas variações:

### 2.2.1 Hill Climbing com Movimentos Laterais

O algoritmo começa com uma configuração inicial aleatória e realiza movimentos laterais, permitindo que as rainhas se movam de maneira que não alterem  $h(n)$ , até um limite de 50 iterações. Essa variação foi criada para escapar de máximos locais sem aumentar o custo computacional.

- **Critério de expansão:** move a rainha apenas dentro da mesma coluna.
- **Estruturas de dados:** vetor de inteiros representando as posições das rainhas.
- **Métricas registradas:** número de iterações, número de conflitos, tempo de execução.

```
std::vector<int> hill_climbing_lateral(int tamanho_tabuleiro, int max_lateral, int *numero_iteracoes) {

    std::vector<int> current_board = initial_board(tamanho_tabuleiro);
    int current_conflicts = conflicts(current_board);
    int lateral_moves = 0;

    *numero_iteracoes = 0;

    while (true) {
        (*numero_iteracoes)++;

        std::vector<Movimento> possible_moves = neighbors(current_board);
        if (possible_moves.empty()) break;

        std::vector<std::vector<int>> neighbor_boards;
        std::vector<int> neighbor_conflicts;

        for (const auto& move : possible_moves) {
            std::vector<int> new_board = apply_board(current_board, move);
            neighbor_boards.push_back(new_board);
            neighbor_conflicts.push_back(conflicts(new_board));
        }

        auto min_it = std::min_element(neighbor_conflicts.begin(), neighbor_conflicts.end());
        int min_conflicts = *min_it;
        std::size_t min_index = std::distance(neighbor_conflicts.begin(), min_it);
        std::vector<int> best_board = neighbor_boards[min_index];

        if (min_conflicts == current_conflicts) {
            if (++lateral_moves > max_lateral) break;
            current_board = best_board;
        } else if (min_conflicts < current_conflicts) {
            current_board = best_board;
            current_conflicts = min_conflicts;
            lateral_moves = 0;
        } else break;

        if (current_conflicts == 0) break;
    }

    return current_board;
}
```

### 2.2.2 Hill Climbing com Reinício Aleatório

O algoritmo de **Hill Climbing com Reinício Aleatório** realiza reinícios sempre que atinge um máximo local. A cada reinício, o algoritmo começa com uma nova configuração aleatória. O número máximo de reinícios foi limitado a 50, e cada execução é limitada a 50 iterações.

- **Critério de reinício:** se o algoritmo não encontrar uma melhoria após  $k$  iterações, reinicia a busca.
- **Estruturas de dados:** vetor de inteiros representando as posições das rainhas.
- **Métricas registradas:** número de reinícios, número total de iterações, tempo de execução.

```
std::vector<int> hill_climbing_random(int tamanho_tabuleiro, int max_attempts, int *numero_iteracoes) {
    std::vector<int> current_board = initial_board(tamanho_tabuleiro);
    int current_conflicts = conflicts(current_board);

    *numero_iteracoes = 0;

    for (int attempt = 0; attempt < max_attempts; ++attempt) {
        (*numero_iteracoes)++;

        std::vector<Movimento> possible_moves = neighbors(current_board);
        if (possible_moves.empty()) break;

        std::vector<std::vector<int>> neighbor_boards;
        std::vector<int> neighbor_conflicts;

        for (const auto& move : possible_moves) {
            std::vector<int> new_board = apply_board(current_board, move);
            neighbor_boards.push_back(new_board);
            neighbor_conflicts.push_back(conflicts(new_board));
        }

        auto min_it = std::min_element(neighbor_conflicts.begin(), neighbor_conflicts.end());
        int min_conflicts = *min_it;
        std::size_t min_index = std::distance(neighbor_conflicts.begin(), min_it);
        std::vector<int> best_board = neighbor_boards[min_index];

        if (min_conflicts < current_conflicts) {
            current_board = best_board;
            current_conflicts = min_conflicts;
        } else {
            int random_index = rand() % possible_moves.size();
            current_board = apply_board(current_board, possible_moves[random_index]);
            current_conflicts = conflicts(current_board);
        }

        if (current_conflicts == 0) break;
    }

    return current_board;
}

std::vector<int> hill_climbing_random_restart(int tamanho_tabuleiro, int max_restarts, int max_attempts,
```

```

int *numero_iteracoes, int *num_reinicios) {
    std::vector<int> best_board;
    int best_conflicts = INT_MAX;

    *numero_iteracoes = 0;
    *num_reinicios = 0;

    for (int restart = 0; restart < max_restarts; ++restart) {
        (*num_reinicios)++;

        int iter_local = 0;
        std::vector<int> solved_board = hill_climbing_random(tamanho_tabuleiro, max_attempts, &iter_local);

        (*numero_iteracoes) += iter_local;

        int current_conflicts = conflicts(solved_board);

        if (current_conflicts < best_conflicts) {
            best_conflicts = current_conflicts;
            best_board = solved_board;
        }

        if (best_conflicts == 0) break;
    }

    return best_board;
}

```

## 2.3 Métricas de Avaliação

As métricas coletadas para a avaliação do algoritmo foram:

- **Tempo de execução (ms):** tempo total até encontrar a solução.
- **Número de iterações:** número total de iterações realizadas pelo algoritmo até encontrar a solução.
- **Número de reinícios (para o algoritmo com reinício aleatório):** número total de reinícios realizados.
- **Número de movimentos:** número total de movimentos realizados pelas rainhas.
- **Taxa de sucesso:** porcentagem de execuções que encontraram a solução.

Essas métricas são utilizadas para comparar o desempenho de ambos os algoritmos: Hill Climbing com Movimentos Laterais e Hill Climbing com Reinício Aleatório.

## 3 Experimentos e Resultados

Foram realizadas 100 execuções independentes, com semente fixa para garantir a reprodutibilidade dos resultados.

| Algoritmo                           | Tempo (ms) | Reinícios | Taxa de Sucesso | Movimentos Médios |
|-------------------------------------|------------|-----------|-----------------|-------------------|
| Hill Climbing (Movimentos Laterais) | 1.02       | 0         | 87%             | 55                |
| Hill Climbing (Reinício Aleatório)  | 1.15       | 2.3       | 100%            | 61                |

Tabela 1: Resultados médios obtidos nas execuções.

## 4 Análise e Discussão

Os resultados obtidos durante a execução dos algoritmos de Hill Climbing com Movimentos Laterais e Hill Climbing com Reinício Aleatório indicam um desempenho satisfatório no contexto do problema das Oito Rainhas. No entanto, as diferenças entre as variações se tornam evidentes quando analisamos métricas como o número de iterações, taxa de sucesso e tempo de execução.

### 4.1 Hill Climbing com Movimentos Laterais

O algoritmo de Hill Climbing com Movimentos Laterais mostrou-se eficiente na busca por uma solução, com uma taxa de sucesso de 87%. Esse algoritmo foi capaz de escapar de alguns platôs rasos devido à possibilidade de realizar movimentos laterais, onde as rainhas se movem dentro da mesma coluna sem alterar o valor de  $h(n)$ . Contudo, essa variação apresentou uma maior quantidade de movimentos médios (55 por execução) e um tempo de execução de 1.02 ms, o que sugere que, embora escape de platôs, o algoritmo ainda não é completamente eficiente em termos de custo computacional.

O número de iterações (50) foi alcançado devido à limitação de movimentos laterais, o que proporcionou uma solução razoável, mas sem garantir a otimalidade. Esse comportamento é esperado, já que o Hill Climbing, por sua natureza, pode facilmente se estagnar em máximos locais, e a variação com movimentos laterais tenta mitigar essa falha sem garantias de optimalidade.

### 4.2 Hill Climbing com Reinício Aleatório

Por outro lado, o Hill Climbing com Reinício Aleatório foi extremamente eficaz, alcançando 100% de taxa de sucesso, como era esperado. Esse algoritmo, ao reiniciar a busca a cada máximo local, garantiu que a solução final fosse encontrada independentemente das dificuldades impostas pelos máximos locais ou platôs. A quantidade de reinícios (média de 2.3 por execução) indica que, embora o algoritmo precise reiniciar algumas vezes, ele é robusto o suficiente para sempre encontrar uma solução.

Em termos de tempo de execução (1.15 ms), o algoritmo apresentou um desempenho ligeiramente inferior ao da versão com movimentos laterais. No entanto, isso é compensado pela maior confiabilidade e pela taxa de sucesso de 100%, o que é uma vantagem considerável em problemas de otimização combinatória, onde a confiabilidade é muitas vezes mais importante do que a eficiência computacional.

A maior quantidade de iterações (61 movimentos médios por execução) reflete a necessidade de reinícios, que aumentam o número de vezes que o algoritmo explora diferentes estados. A vantagem é que essa abordagem garante uma solução ótima (sem conflitos entre as rainhas), ao contrário da variação com movimentos laterais, que não assegura uma solução ótima em todos os casos.

## 5 Conclusão

Em resumo, os resultados obtidos indicam que a escolha entre as duas variações do algoritmo de Hill Climbing depende dos critérios de desempenho adotados para o problema das 8 Rainhas.

- O Hill Climbing com Movimentos Laterais mostrou-se uma abordagem útil para escapar de máximos locais, mas apresenta limitações em termos de eficiência computacional e não garante a otimalidade. O aumento no número de movimentos médios e a execução mais rápida tornam esta versão interessante para cenários onde a velocidade é mais importante que a garantia de uma solução ótima.

- O Hill Climbing com Reinício Aleatório foi a solução mais robusta, com 100% de sucesso, garantindo a solução ótima a todo momento. Embora tenha apresentado um custo computacional maior devido ao número de reinícios e movimentos, essa abordagem se destacou pela confiabilidade e por não ser vulnerável a máximos locais. Para problemas em que a confiabilidade e a solução ótima são prioritárias, essa variação mostrou-se mais adequada.

Portanto, a escolha do algoritmo ideal depende da prioridade entre eficiência computacional e confiabilidade. A abordagem com reinício aleatório se mostrou mais adequada para problemas onde a solução correta é imprescindível, enquanto o método com movimentos laterais pode ser uma boa escolha quando a eficiência computacional é mais importante, mesmo que isso signifique comprometer a solução ótima em alguns casos.

## Créditos e Declaração de Autoria

**Autor:** Anderson Rodrigues dos Santos

**Algoritmos implementados:** Hill Climbing com Movimentos Laterais e com Re-início Aleatório

**Uso de IA:** ChatGPT (OpenAI) utilizado para tirar dúvidas sobre estruturas de conceitos. Copilot (GitHub) auxílio na programação.

**Recursos externos:** notas de aula da disciplina, exemplos fornecidos pelo professor e materiais de apoio disponíveis do curso.

**Autora:** Maíra Beatriz de Almeida Lacerda

**Responsável por:** Documentação, escrita do README.md e formatação em  $\text{\LaTeX}$ .

**Uso de IA:** ChatGPT (OpenAI) utilizado exclusivamente para revisão textual e formatação em  $\text{\LaTeX}$ .

## Referências

1. RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4<sup>a</sup> ed. Pearson, 2022.
2. OLIVEIRA, Tiago A. *Inteligência Artificial – Engenharia de Computação (EC\_IA\_005)*. CEFET-MG, 2025.
3. PEARL, J. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
4. ANDERSON SANTOS. *Inteligência Artificial — Busca e 8 Rainhas*. Repositório GitHub: [https://github.com/anderrasantos/Inteligencia\\_Artificial\\_Busca\\_e\\_8\\_Rainhas](https://github.com/anderrasantos/Inteligencia_Artificial_Busca_e_8_Rainhas).